

Álgebra Lineal Computacional con Python

Fundamentos Teóricos, Algoritmos con `numpy.linalg` y Aplicaciones Prácticas

Documento de Referencia Avanzada

Departamento de Ciencias de la Computación y Matemáticas Aplicadas

17 de septiembre de 2026

Resumen

Este documento constituye un manual exhaustivo y riguroso para la implementación de algoritmos de álgebra lineal computacional en Python, tomando como base y extendiendo significativamente los temas abordados en las guías canónicas de `numpy.linalg` y `scipy.linalg`. Abarca desde la estructuración de memoria y manipulación de tensores en $\mathbb{R}^n$ y $\mathbb{R}^{m \times n}$, hasta descomposiciones factoriales fundamentales (LU, Cholesky, QR, SVD), resolución estable de sistemas de ecuaciones mal condicionados, análisis de valores y vectores propios, y ajuste por mínimos cuadrados. Cada sección articula el fundamento matemático formal, consideraciones de estabilidad numérica y código fuente moderno con sus correspondientes salidas interpretadas, incorporando además navegación bidireccional interactiva hacia el índice general.

Índice

1. Introducción al Cómputo Matricial en Python	2
1.1. Arquitectura Interna: NumPy, BLAS y LAPACK	2
1.2. Instalación del Entorno Numérico	2
1.3. Convenciones de Importación Estándar	3
2. Operaciones Básicas con Vectores y Matrices	3
2.1. Definición de Vectores y Matrices en $\mathbb{R}^n$ y $\mathbb{R}^{m \times n}$	3
2.1.1. Matrices Especiales y Generadores de Rango	4
2.2. Producto Punto, Producto Hadamard y Multiplicación Matricial	4
3. Operaciones Avanzadas con <code>numpy.linalg</code>	5
3.1. Determinante de una Matriz Cuadrada	5
3.2. Matriz Inversa	5
3.3. Solución Numérica de Sistemas Lineales	6
3.3.1. El Número de Condición (<i>Condition Number</i>)	7
3.4. Valores y Vectores Propios (Diagonalización)	7
3.5. Normas Vectoriales y Matriciales	8
3.5.1. Normas en Vectores $\mathbf{v} \in \mathbb{R}^n$	8
3.5.2. Normas en Matrices $A \in \mathbb{R}^{m \times n}$	8
3.6. Rango Matricial y Espacios Fundamentales	9
4. Descomposiciones Matriciales Avanzadas	9
4.1. Descomposición en Valores Singulares (SVD)	9
4.2. Descomposición QR	11
4.3. Descomposición de Cholesky	11
4.4. Factorización LU con Pivoteo Parcial	12

5. Mínimos Cuadrados y Pseudoinversa	13
5.1. Sistemas Sobredeterminados y Mínimos Cuadrados (<code>la.lstsq</code>)	13
5.2. Pseudoinversa de Moore-Penrose (<code>la.pinv</code>)	14
6. Ejemplo Integrador Completo: Calibración y Sensibilidad	14
7. Buenas Prácticas, Rendimiento y Estabilidad Numérica	16
7.1. ¿Cuándo Utilizar <code>numpy.linalg</code> frente a <code>scipy.linalg</code> ?	16
7.2. Disposición de Datos en Memoria: Orden C vs. Orden Fortran	16
7.3. Comparaciones Numéricas Seguras	16
8. Tabla Resumen de Funciones y Complejidades	17
9. Conclusiones y Referencias	18
9.1. Conclusiones	18
9.2. Referencias Bibliográficas	18

1. Introducción al Cómputo Matricial en Python

El álgebra lineal constituye el lenguaje matemático fundamental del cómputo científico moderno, sustentando campos como el aprendizaje automático (*Machine Learning*), la visión por computador, la optimización convexa, el análisis de señales y la robótica. En Python, la arquitectura para operaciones con tensores descansa en **NumPy** (*Numerical Python*) y **SciPy**.

1.1. Arquitectura Interna: NumPy, BLAS y LAPACK

A diferencia de los tipos de datos nativos de Python (como listas o tuplas), que introducen sobrecarga por verificación dinámica de tipos y falta de localidad espacial en memoria, los arreglos de NumPy (`np.ndarray`) se almacenan en bloques de memoria contigua en lenguaje C o Fortran.

Aceleración mediante Bibliotecas de Bajo Nivel

Las operaciones del submódulo `numpy.linalg` no están programadas en Python puro. Delegan su cálculo directamente a bibliotecas altamente optimizadas a nivel de ensamblador y procesador:

- **BLAS** (*Basic Linear Algebra Subprograms*): Niveles 1 (vector-vector), 2 (matriz-vector) y 3 (matriz-matriz), optimizados con instrucciones vectoriales SIMD (AVX-512, NEON).
- **LAPACK** (*Linear Algebra PACKage*): Rutinas de factorización matricial, cálculo de autovalores y resolución de sistemas lineales mediante eliminación gaussiana por bloques.

1.2. Instalación del Entorno Numérico

Para instalar el ecosistema estándar recomendado para cálculo científico:

```

1 # Instalación via gestor pip
2 pip install numpy scipy matplotlib
3
4 # O mediante entornos reproducibles Conda / Mamba
5 conda install -c conda-forge numpy scipy matplotlib

```

Listing 1: Comandos para instalación de dependencias en terminal

1.3. Convenciones de Importación Estándar

En la comunidad científica y en el desarrollo profesional, se emplean los siguientes alias canónicos:

```
1 import numpy as np           # Manipulación de arreglos n-dimensionales
2 import numpy.linalg as la    # Submodulo estandar de algebra lineal
3 import scipy.linalg as spla  # Modulo extendido para rutinas avanzadas LAPACK
```

Listing 2: Importación estándar de librerías numéricas

▲ Volver al índice general

2. Operaciones Básicas con Vectores y Matrices

2.1. Definición de Vectores y Matrices en $\mathbb{R}^n$ y $\mathbb{R}^{m \times n}$

Un vector columna $\mathbf{x} \in \mathbb{R}^n$ y una matriz $A \in \mathbb{R}^{m \times n}$ se expresan formalmente como:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \quad A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} \quad (1)$$

En NumPy, los vectores unidimensionales tienen forma $(n,)$, mientras que las matrices son arreglos 2D con forma (m, n) . Para formular vectores columna estrictos en álgebra matricial 2D, se especifica una forma $(n, 1)$.

```
1 import numpy as np
2
3 # 1. Vector 1D en R^3
4 v = np.array([1.0, 2.0, 3.0], dtype=np.float64)
5
6 # 2. Matriz 2x2 en R^(2x2)
7 A = np.array([[4.0, 2.0],
8               [1.0, 3.0]], dtype=np.float64)
9
10 # 3. Vector columna formal 2D (2x1)
11 b = np.array([[10.0],
12               [8.0]], dtype=np.float64)
13
14 print(f"Dimension v: {v.shape}, Tipo: {v.dtype}")
15 print(f"Dimension A: {A.shape}, Numero de elementos: {A.size}")
16 print(f"Dimension b (columna): {b.shape}")
```

Listing 3: Creación de vectores, matrices y verificación de atributos

Consola de Salida: Atributos de arreglos

```
Dimension v: (3,), Tipo: float64
Dimension A: (2, 2), Numero de elementos: 4
Dimension b (columna): (2, 1)
```

2.1.1. Matrices Especiales y Generadores de Rango

NumPy incluye constructores optimizados para instanciar matrices de uso frecuente sin recurrir a bucles:

```
1 I3 = np.eye(3)                # Matriz identidad 3x3
2 Z23 = np.zeros((2, 3))        # Matriz nula 2x3
3 O33 = np.ones((3, 3))         # Matriz de unos 3x3
4 D = np.diag([5.0, -2.0, 3.0]) # Matriz diagonal
5 lin = np.linspace(0, 1, 5)     # Vector con 5 puntos equiespaciados en [0, 1]
```

Listing 4: Generación de matrices canónicas

[▲ Volver al índice general](#)

2.2. Producto Punto, Producto Hadamard y Multiplicación Matricial

Una fuente habitual de errores conceptuales en Python radica en confundir la multiplicación elemento a elemento con el producto matricial.

¡Atención Crítica! — Distinción Operacional Clave

- **A * B: Producto de Hadamard** (elemento a elemento). Requiere que ambas matrices posean dimensiones idénticas o difusibles (*broadcasting*). $C_{ij} = A_{ij} \cdot B_{ij}$.
- **A @ B o np.matmul(A, B): Multiplicación Matricial estándar** de álgebra lineal. Requiere que el número de columnas del primer operando coincida con las filas del segundo ($A \in \mathbb{R}^{m \times k}, B \in \mathbb{R}^{k \times n} \implies C \in \mathbb{R}^{m \times n}$).
- **np.dot(u, v):** En arreglos 1D calcula el producto interno euclidiano $\mathbf{u}^T \mathbf{v} = \sum_{i=1}^n u_i v_i$.

```
1 # Definición de vectores y matrices
2 u = np.array([1.0, 2.0, 3.0])
3 v = np.array([4.0, 5.0, 6.0])
4
5 # Producto punto entre vectores
6 dot_prod = np.dot(u, v)      # O alternativamente: u @ v
7 print(f"Producto punto u . v = {dot_prod}")
8
9 # Matrices conformables para multiplicación
10 M1 = np.array([[1.0, 2.0],
11               [3.0, 4.0]])
12 M2 = np.array([[5.0, 6.0],
13               [7.0, 8.0]])
14
15 # Producto Hadamard vs Producto Matricial
16 hadamard = M1 * M2
17 matricial = M1 @ M2          # Equivalente a np.matmul(M1, M2)
18
19 print("Producto de Hadamard (M1 * M2):\n", hadamard)
20 print("Producto Matricial (M1 @ M2):\n", matricial)
```

Listing 5: Demostración de productos matriciales y vectoriales

Consola de Salida: Comparativa de productos

```
Producto punto u . v = 32.0
Producto de Hadamard (M1 * M2):
[[ 5. 12.]
 [21. 32.]]
```

```
Producto Matricial (M1 @ M2):
[[19. 22.]
 [43. 50.]]
```

[▲ Volver al índice general](#)

3. Operaciones Avanzadas con numpy.linalg

El submódulo `numpy.linalg` implementa los algoritmos canónicos de álgebra lineal numérica sobre matrices cuadradas y rectangulares.

3.1. Determinante de una Matriz Cuadrada

Para una matriz cuadrada $A \in \mathbb{R}^{n \times n}$, el determinante $\det(A)$ mide el factor de escala con el que la transformación lineal altera el volumen n -dimensional de cualquier región geométrica. Su signo indica si la transformación preserva o invierte la orientación del espacio.

$$\det \begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc \quad (2)$$

```
1 A = np.array([[4.0, 2.0],
2               [1.0, 3.0]])
3
4 det_A = la.det(A)
5 print(f"det(A) = {det_A:.6f}")
```

Listing 6: Cálculo del determinante con `la.det`

Consola de Salida: Determinante

```
det(A) = 10.000000
```

Recomendación de Rendimiento: Prevención de Desbordamiento (*Overflow/Underflow*) con `la.slogdet`

Para matrices de gran escala ($n > 50$), el determinante numérico directo suele desbordarse a $\pm\infty$ o 0.0 debido al producto de sus valores singulares. Se recomienda utilizar `sign`, `logdet` = `la.slogdet(A)`, que calcula de manera numéricamente estable el signo y el logaritmo natural del determinante: $\det(A) = \text{sign} \cdot \exp(\text{logdet})$.

[▲ Volver al índice general](#)

3.2. Matriz Inversa

Una matriz cuadrada A admite una inversa multiplicativa A^{-1} tal que:

$$AA^{-1} = A^{-1}A = I_n \iff \det(A) \neq 0 \quad (3)$$

```

1  # Obtencion de la inversa
2  A_inv = la.inv(A)
3
4  print("Matriz inversa A^(-1):\n", A_inv)
5
6  # Verificacion numerica de idempotencia: A @ A_inv ~ I
7  identidad_aprox = A @ A_inv
8  es_identidad = np.allclose(identidad_aprox, np.eye(2))
9
10 print("Comprobacion A @ A^(-1):\n", np.round(identidad_aprox, 6))
11 print(f"Es estrictamente la matriz identidad dentro de tolerancia: {es_identidad}")

```

Listing 7: Cálculo y verificación formal de la matriz inversa

Consola de Salida: Inversa calculada

```

Matriz inversa A^(-1):
[[ 0.3 -0.2]
 [-0.1  0.4]]
Comprobacion A @ A^(-1):
[[1. 0.]
 [0. 1.]]
Es estrictamente la matriz identidad dentro de tolerancia: True

```

¡Atención Crítica! — Por qué nunca debe invertirse una matriz para resolver $Ax = b$

En códigos de producción, jamás debe computarse $x = A^{-1}b$ calculando primero `la.inv(A)`. Calcular la inversa explícita requiere $O(n^3)$ operaciones con una constante asintótica elevada y duplica la susceptibilidad al error por acumulación de redondeo de punto flotante. Siempre se debe utilizar `la.solve(A, b)`.

[▲ Volver al índice general](#)**3.3. Solución Numérica de Sistemas Lineales**

Un sistema de n ecuaciones lineales con n incógnitas se escribe en forma matricial compacta como:

$$Ax = b \iff \begin{bmatrix} 4 & 2 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 10 \\ 8 \end{bmatrix} \quad (4)$$

La función `la.solve(A, b)` resuelve este sistema mediante la factorización $PA = LU$ (con pivoteo parcial) ejecutada en rutinas LAPACK `_gesv`, alcanzando máxima velocidad y precisión.

```

1  A = np.array([[4.0, 2.0],
2                [1.0, 3.0]])
3  b = np.array([10.0, 8.0])
4
5  # Solucion directa y estable
6  x = la.solve(A, b)
7
8  print(f"Solucion calculada x = {x}")
9
10 # Verificacion del residuo euclidiano: ||Ax - b||_2
11 residuo = la.norm(A @ x - b)
12 print(f"Norma L2 del vector residuo ||Ax - b|| = {residuo:.2e}")

```

Listing 8: Resolución eficiente de sistemas lineales con `la.solve`

Consola de Salida: Solución del sistema

```
Solucion calculada x = [1.4 2.2]
Norma L2 del vector residuo ||Ax - b|| = 0.00e+00
```

3.3.1. El Número de Condición (*Condition Number*)

El número de condición $\kappa(A) = \|A\| \cdot \|A^{-1}\|$ cuantifica la sensibilidad de la solución $\mathbf{x}$ ante perturbaciones infinitesimales en el vector $\mathbf{b}$ o en los coeficientes de A :

$$\frac{\|\Delta \mathbf{x}\|}{\|\mathbf{x}\|} \leq \kappa(A) \frac{\|\Delta \mathbf{b}\|}{\|\mathbf{b}\|} \quad (5)$$

En NumPy se calcula con `la.cond(A)`. Si $\kappa(A) \approx 1$, la matriz está perfectamente condicionada. Si $\kappa(A) \gg 10^{12}$, la matriz es cuasi-singular y se perderán dígitos significativos de precisión en la solución.

[▲ Volver al índice general](#)

3.4. Valores y Vectores Propios (Diagonalización)

Sea $A \in \mathbb{R}^{n \times n}$. Un escalar $\lambda \in \mathbb{C}$ y un vector no nulo $\mathbf{v} \neq \mathbf{0}$ son respectivamente un valor propio (*eigenvalue*) y un vector propio (*eigenvector*) si satisfacen:

$$A\mathbf{v} = \lambda\mathbf{v} \iff (A - \lambda I)\mathbf{v} = \mathbf{0} \quad (6)$$

Geométricamente, la acción lineal de A sobre el autovector $\mathbf{v}$ no altera su dirección en el espacio vectorial; únicamente escala su magnitud por el factor λ .

```
1 A = np.array([[4.0, 2.0],
2               [1.0, 3.0]])
3
4 valores, vectores = la.eig(A)
5
6 print("Valores propios (lambda_i):", valores)
7 print("Matriz modal de autovectores (columnas):\n", vectores)
8
9 # Comprobacion de la definicion A v_i = lambda_i v_i
10 for i in range(len(valores)):
11     lam = valores[i]
12     v_i = vectores[:, i]
13     err = la.norm(A @ v_i - lam * v_i)
14     print(f"Autovalor {i+1} = {lam:.4f} | Error ||A v - lambda v|| = {err:.2e}")
```

Listing 9: Cálculo de autovalores y autovectores con `la.eig`

Consola de Salida: Autovalores y autovectores

```
Valores propios (lambda_i): [5. 2.]
Matriz modal de autovectores (columnas):
[[ 0.89442719 -0.70710678]
 [ 0.4472136  0.70710678]]
Autovalor 1 = 5.0000 | Error ||A v - lambda v|| = 0.00e+00
Autovalor 2 = 2.0000 | Error ||A v - lambda v|| = 0.00e+00
```

Recomendación de Rendimiento: Matrices Simétricas / Hermíticas: Emplee siempre `la.eigh`

Si se garantiza teóricamente que la matriz es simétrica real ($A = A^T$), utilice `la.eigh(A)`. El algoritmo `eigh` está basado en transformaciones de Householder y el algoritmo QR simétrico, asegurando que todos los autovalores sean estrictamente reales, garantizando la ortogonalidad mutua de los autovectores y duplicando la velocidad de convergencia.

[▲ Volver al índice general](#)

3.5. Normas Vectoriales y Matriciales

La norma cuantifica la magnitud métrica o longitud de un elemento en su espacio vectorial. Se evalúa con `la.norm(x, ord=...)`.

3.5.1. Normas en Vectores $\mathbf{v} \in \mathbb{R}^n$

- **Norma Euclidiana (L_2)** (`ord=2` o por defecto): $\|\mathbf{v}\|_2 = \sqrt{\sum_{i=1}^n |v_i|^2}$.
- **Norma Manhattan (L_1)** (`ord=1`): $\|\mathbf{v}\|_1 = \sum_{i=1}^n |v_i|$.
- **Norma del Máximo (L_∞)** (`ord=np.inf`): $\|\mathbf{v}\|_\infty = \max_i |v_i|$.

3.5.2. Normas en Matrices $A \in \mathbb{R}^{m \times n}$

- **Norma de Frobenius** (`ord='fro'`): $\|A\|_F = \sqrt{\sum_{i,j} |a_{ij}|^2} = \sqrt{\text{tr}(A^T A)}$.
- **Norma Espectral / Inducida L_2** (`ord=2`): $\|A\|_2 = \sigma_{\max}(A)$ (el mayor valor singular).

```

1 v = np.array([3.0, 4.0])
2
3 norma_l2 = la.norm(v)           # Por defecto: L2
4 norma_l1 = la.norm(v, ord=1)    # L1
5 norma_inf = la.norm(v, ord=np.inf) # Linf
6
7 print(f"Vector: {v}")
8 print(f"Norma L2 (Euclidiana) : {norma_l2}")
9 print(f"Norma L1 (Manhattan)   : {norma_l1}")
10 print(f"Norma L_inf (Maximo)   : {norma_inf}")
11
12 # Normas matriciales sobre matriz 2x2
13 A_mat = np.array([[1.0, 2.0],
14                   [3.0, 4.0]])
15 print(f"Norma de Frobenius ||A||_F: {la.norm(A_mat, ord='fro'):.4f}")
16 print(f"Norma inducida 2 ||A||_2 : {la.norm(A_mat, ord=2):.4f}")

```

Listing 10: Cálculo de diversas normas con `la.norm`

Consola de Salida: Normas calculadas

```

Vector: [3. 4.]
Norma L2 (Euclidiana) : 5.0
Norma L1 (Manhattan)   : 7.0
Norma L_inf (Maximo)   : 4.0
Norma de Frobenius ||A||_F: 5.4772
Norma inducida 2 ||A||_2 : 5.4650

```

[▲ Volver al índice general](#)

3.6. Rango Matricial y Espacios Fundamentales

El rango $\text{rang}(A)$ corresponde al número máximo de filas o columnas linealmente independientes de A . Determina la dimensión del espacio columna $\text{Col}(A)$ y del espacio fila $\text{Fil}(A)$.

Determinación Numérica del Rango vía SVD

NumPy no calcula el rango mediante eliminación gaussiana ingenua, debido a que los errores de redondeo generarían elementos no nulos espurios. En su lugar, `la.matrix_rank(A)` evalúa los valores singulares $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_k$ y cuenta cuántos de ellos superan una tolerancia numérica relativa:

$$\text{tol} = \sigma_{\text{máx}} \cdot \text{máx}(m, n) \cdot \varepsilon_{\text{machine}} \quad (7)$$

```
1 # Matriz 3x3 donde la Fila 3 = 2*Fila 2 - Fila 1 (linealmente dependiente)
2 A_dep = np.array([[1.0, 2.0, 3.0],
3                  [4.0, 5.0, 6.0],
4                  [7.0, 8.0, 9.0]])
5
6 rango = la.matrix_rank(A_dep)
7 print(f"Matriz:\n{A_dep}")
8 print(f"Rango de la matriz (filas independientes): {rango} (de 3 posibles)")
```

Listing 11: Cálculo del rango en matrices de rango deficiente

Consola de Salida: Rango numérico

```
Matriz:
[[1. 2. 3.]
 [4. 5. 6.]
 [7. 8. 9.]]
Rango de la matriz (filas independientes): 2 (de 3 posibles)
```

[▲ Volver al índice general](#)

4. Descomposiciones Matriciales Avanzadas

Las factorizaciones o descomposiciones matriciales descomponen una matriz compleja en el producto de matrices estructuralmente canónicas (ortogonales, triangulares o diagonales), simplificando drásticamente el análisis teórico y la resolución algorítmica.

4.1. Descomposición en Valores Singulares (SVD)

El Teorema de Descomposición en Valores Singulares establece que **cualquier** matriz rectangular $A \in \mathbb{R}^{m \times n}$ puede descomponerse en el producto:

$$A = U \Sigma V^T \quad (8)$$

donde:

- $U \in \mathbb{R}^{m \times m}$ es una matriz ortogonal ($U^T U = I_m$), cuyas columnas son los vectores singulares izquierdos (autovectores de AA^T).

- $\Sigma \in \mathbb{R}^{m \times n}$ es una matriz pseudodiagonal que contiene los valores singulares ordenados de mayor a menor: $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_{\min(m,n)} \geq 0$.
- $V \in \mathbb{R}^{n \times n}$ es una matriz ortogonal ($V^T V = I_n$), cuyas columnas son los vectores singulares derechos (autovectores de $A^T A$).

```

1 A = np.array([[1.0, 2.0],
2               [3.0, 4.0],
3               [5.0, 6.0]]) # Matriz 3x2
4
5 # SVD con numpy.linalg
6 U, s, Vt = la.svd(A, full_matrices=False) # Forma reducida / economizada
7
8 print("Matriz U (ortogonal izquierda):\n", np.round(U, 4))
9 print("Valores singulares sigma:", np.round(s, 4))
10 print("Matriz V^T (ortogonal derecha):\n", np.round(Vt, 4))
11
12 # Reconstruccion de A = U @ Sigma @ V^T
13 Sigma = np.diag(s)
14 A_reconstruida = U @ Sigma @ Vt
15
16 print("A reconstruida:\n", np.round(A_reconstruida, 4))
17 print(f"Error de reconstruccion: {la.norm(A - A_reconstruida):.2e}")

```

Listing 12: Descomposición SVD completa y reconstrucción matricial

Consola de Salida: Salida SVD

```

Matriz U (ortogonal izquierda):
[[-0.2298  0.8835]
 [-0.5247  0.2408]
 [-0.8196 -0.4019]]
Valores singulares sigma: [9.5255 0.5143]
Matriz V^T (ortogonal derecha):
[[-0.6196 -0.7849]
 [-0.7849  0.6196]]
A reconstruida:
[[1. 2.]
 [3. 4.]
 [5. 6.]]
Error de reconstruccion: 0.00e+00

```

Teorema de Eckart-Young-Mirsky (Compresión y Reducción de Ruido)

El SVD proporciona la mejor aproximación matricial de bajo rango $k < r$ en el sentido de la norma de Frobenius y la norma espectral:

$$A_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^T = \arg \min_{\text{rang}(B) \leq k} \|A - B\|_F \quad (9)$$

Este principio sustenta el Análisis de Componentes Principales (PCA), la compresión de imágenes, el filtrado colaborativo en sistemas de recomendación y el procesamiento de lenguaje natural.

[▲ Volver al índice general](#)

4.2. Descomposición QR

Toda matriz $A \in \mathbb{R}^{m \times n}$ con $m \geq n$ puede factorizarse como:

$$A = QR \quad (10)$$

donde $Q \in \mathbb{R}^{m \times m}$ tiene columnas ortonormales ($Q^T Q = I$) y $R \in \mathbb{R}^{m \times n}$ es una matriz triangular superior.

NumPy incluye `la.qr` de manera nativa (sin requerir SciPy), si bien el módulo `scipy.linalg.qr` provee descomposiciones adicionales con pivoteo.

```

1 A = np.array([[1.0, 2.0],
2               [3.0, 4.0],
3               [5.0, 6.0]])
4
5 # Descomposicion QR reducida (economica: Q de 3x2, R de 2x2)
6 Q, R = la.qr(A, mode='reduced')
7
8 print("Matriz Q (columnas ortonormales):\n", np.round(Q, 4))
9 print("Matriz R (triangular superior):\n", np.round(R, 4))
10
11 # Verificacion: Q^T @ Q = I_2
12 print("Comprobacion Q^T @ Q:\n", np.round(Q.T @ Q, 6))
13 # Verificacion: Q @ R = A
14 print("Comprobacion Q @ R == A:\n", np.allclose(Q @ R, A))

```

Listing 13: Factorización QR en modo económico y verificación de ortogonalidad

Consola de Salida: Factorización QR

```

Matriz Q (columnas ortonormales):
[[-0.169  0.8971]
 [-0.5071 0.276 ]
 [-0.8452 -0.345 ]]
Matriz R (triangular superior):
[[-5.9161 -7.4374]
 [ 0.      0.8281]]
Comprobacion Q^T @ Q:
[[1. 0.]
 [0. 1.]]
Comprobacion Q @ R == A: True

```

[▲ Volver al índice general](#)

4.3. Descomposición de Cholesky

Para matrices cuadradas simétricas y **definidas positivas** ($A = A^T$ y $\mathbf{x}^T A \mathbf{x} > 0$ para todo $\mathbf{x} \neq \mathbf{0}$), la descomposición de Cholesky factoriza A como:

$$A = LL^T \quad (11)$$

donde L es una matriz triangular inferior con entradas diagonales estrictamente positivas.

```

1 # Matriz simetrica definida positiva: K = B^T @ B + alpha*I
2 B = np.array([[2.0, -1.0],
3               [1.0,  3.0]])
4 K = B.T @ B
5
6 # Factorizacion de Cholesky
7 L = la.cholesky(K)

```

```

8
9 print("Matriz original definida positiva K:\n", K)
10 print("Factor triangular inferior L:\n", np.round(L, 4))
11 print("Comprobacion L @ L^T == K:\n", np.allclose(L @ L.T, K))

```

Listing 14: Descomposición de Cholesky con `la.cholesky`**Consola de Salida: Cholesky**

```

Matriz original definida positiva K:
[[ 5.  1.]
 [ 1. 10.]]
Factor triangular inferior L:
[[2.2361 0.    ]
 [0.4472 3.1305]]
Comprobacion L @ L^T == K: True

```

Recomendación de Rendimiento: Eficiencia Máxima con Cholesky

La factorización de Cholesky requiere exactamente la mitad de operaciones en punto flotante ($\approx \frac{1}{3}n^3$ flops) que la factorización LU general ($\approx \frac{2}{3}n^3$ flops) y es incondicionalmente estable sin necesidad de pivoteo. En procesos gaussianos, optimización cuadrática y filtros de Kalman, siempre que la matriz de covarianza sea simétrica definida positiva, Cholesky es la opción óptima.

[▲ Volver al índice general](#)**4.4. Factorización LU con Pivoteo Parcial**

Para cualquier matriz cuadrada invertible, la eliminación gaussiana con intercambio de filas produce:

$$PA = LU \quad (12)$$

donde P es una matriz de permutación, L es triangular inferior unitaria (unos en la diagonal) y U es triangular superior.

```

1 A_sq = np.array([[2.0, 1.0, 5.0],
2                 [4.0, 4.0, -4.0],
3                 [1.0, 3.0, 1.0]])
4
5 # Descomposicion PLU con SciPy
6 P, L, U = spla.lu(A_sq)
7
8 print("Matriz L (triangular inferior unitaria):\n", np.round(L, 3))
9 print("Matriz U (triangular superior):\n", np.round(U, 3))
10 print("Comprobacion P @ L @ U == A_sq:", np.allclose(P @ L @ U, A_sq))

```

Listing 15: Factorización LU utilizando `scipy.linalg.lu`**Consola de Salida: Factorización LU**

```

Matriz L (triangular inferior unitaria):
[[ 1.  0.  0. ]
 [ 0.5  1.  0. ]
 [ 0.25 0.8  1. ]]
Matriz U (triangular superior):
[[ 4.  4. -4. ]
 [ 0. -1.  7. ]

```

```
[ 0.  0. -3.6]]
Comprobacion P @ L @ U == A_sq: True
```

[▲ Volver al índice general](#)

5. Mínimos Cuadrados y Pseudoinversa

5.1. Sistemas Sobredeterminados y Mínimos Cuadrados (la.lstsq)

Cuando un sistema $A\mathbf{x} = \mathbf{b}$ tiene más ecuaciones independientes que incógnitas ($m > n$), comúnmente no existe un vector $\mathbf{x}$ que satisfaga todas las ecuaciones simultáneamente. El método de **mínimos cuadrados** busca el vector $\mathbf{x}^*$ que minimiza la norma euclidiana al cuadrado del residuo:

$$\mathbf{x}^* = \arg \min_{\mathbf{x} \in \mathbb{R}^n} \|A\mathbf{x} - \mathbf{b}\|_2^2 \quad (13)$$

La condición de optimalidad conduce a las **ecuaciones normales**:

$$A^T A \mathbf{x} = A^T \mathbf{b} \implies \mathbf{x} = (A^T A)^{-1} A^T \mathbf{b} \quad (14)$$

¡Atención Crítica! — Peligro de las Ecuaciones Normales en Código Real

Construir y resolver directamente $A^T A \mathbf{x} = A^T \mathbf{b}$ eleva al cuadrado el número de condición de la matriz: $\kappa(A^T A) = (\kappa(A))^2$. Por ende, si $\kappa(A) = 10^5$, entonces $\kappa(A^T A) = 10^{10}$, perdiendo instantáneamente la mayor parte de la precisión numérica. La función `la.lstsq` de NumPy utiliza internamente la descomposición SVD o QR con reflexiones de Householder, evitando calcular $A^T A$ y preservando la estabilidad.

```
1 # Datos experimentales: 4 puntos (x_i, y_i)
2 # Buscamos ajustar una recta: y = m*x + c
3 x_puntos = np.array([0.0, 1.0, 2.0, 3.0])
4 y_puntos = np.array([1.1, 1.9, 3.2, 3.8])
5
6 # Construccion de la matriz de diseño A = [x, 1]
7 A_diseno = np.vstack([x_puntos, np.ones(len(x_puntos))]).T
8
9 # Resolucion robusta (rcond=None para fijar el corte SVD segun precision maquina)
10 sol, residuos, rango, s = la.lstsq(A_diseno, y_puntos, rcond=None)
11 pendiente, intercepto = sol
12
13 print(f"Ecuacion ajustada: y = {pendiente:.4f} * x + {intercepto:.4f}")
14 print(f"Valores singulares de la matriz de diseno: {np.round(s, 4)}")
15 print(f"Norma del residuo de ajuste ||Ax - b||: {np.sqrt(residuos[0]):.4f}")
```

Listing 16: Solución robusta por mínimos cuadrados con `la.lstsq`

Consola de Salida: Ajuste por mínimos cuadrados

```
Ecuacion ajustada: y = 0.9400 * x + 1.0900
Valores singulares de la matriz de diseno: [4.1688 1.1399]
Norma del residuo de ajuste ||Ax - b||: 0.1897
```

[▲ Volver al índice general](#)

5.2. Pseudoinversa de Moore-Penrose (la.pinv)

Para cualquier matriz $A \in \mathbb{R}^{m \times n}$ (sea cuadrada o rectangular, de rango completo o deficiente), existe una única matriz $A^+ \in \mathbb{R}^{n \times m}$ que satisface las cuatro ecuaciones de Penrose:

1. $AA^+A = A$
2. $A^+AA^+ = A^+$
3. $(AA^+)^T = AA^+$
4. $(A^+A)^T = A^+A$

Calculada a través de la descomposición SVD ($A = U\Sigma V^T \implies A^+ = V\Sigma^+U^T$, donde los elementos no nulos de Σ se invierten: σ_i^{-1}), permite encontrar la solución de norma mínima a cualquier sistema lineal: $\mathbf{x} = A^+\mathbf{b}$.

```

1 A_rect = np.array([[1.0, 2.0],
2                   [3.0, 4.0],
3                   [5.0, 6.0]])
4
5 # Pseudoinversa
6 A_plus = la.pinv(A_rect)
7
8 print("Pseudoinversa A^+ (dimension 2x3):\n", np.round(A_plus, 4))
9
10 # Verificacion de la 1ra condicion de Penrose: A @ A^+ @ A == A
11 cond1 = A_rect @ A_plus @ A_rect
12 print("Cumple A @ A^+ @ A == A:", np.allclose(cond1, A_rect))

```

Listing 17: Cálculo y verificación de la pseudoinversa de Moore-Penrose

Consola de Salida: Pseudoinversa calculada

```

Pseudoinversa A^+ (dimension 2x3):
[[-1.3333 -0.3333  0.6667]
 [ 1.0833  0.3333 -0.4167]]
Cumple A @ A^+ @ A == A: True

```

[▲ Volver al índice general](#)

6. Ejemplo Integrador Completo: Calibración y Sensibilidad

Para ilustrar la sinergia entre las herramientas estudiadas, abordaremos un problema integral de ingeniería: la identificación experimental de los parámetros de un modelo cuadrático de segundo orden sujeto a ruido gaussiano:

$$y(t) = c_0 + c_1t + c_2t^2 + \varepsilon(t) \quad (15)$$

El script ejecuta los siguientes pasos computacionales:

1. Construcción de la matriz de Vandermonde generalizada $A \in \mathbb{R}^{m \times 3}$.
2. Resolución por mínimos cuadrados mediante `la.lstsq`.
3. Evaluación comparativa del número de condición de A frente a la matriz de las ecuaciones normales A^TA .
4. Cálculo de la matriz de covarianza de los parámetros estimados $\text{Cov}(\mathbf{c}) = \sigma^2(A^TA)^{-1}$.

5. Evaluación de la bondad de ajuste mediante el coeficiente de determinación R^2 .

```

1 import numpy as np
2 import numpy.linalg as la
3
4 # Semilla reproducible para simulacion de mediciones de laboratorio
5 np.random.seed(42)
6
7 # Parametros teoricos exactos
8 c_teorico = np.array([2.5, -1.8, 0.75])
9
10 # Generacion de 8 puntos temporales t en [0, 4]
11 t = np.linspace(0.0, 4.0, 8)
12
13 # Matriz de diseno (Vandermonde: 1, t, t^2)
14 A = np.column_stack([np.ones_like(t), t, t**2])
15
16 # Mediciones sinteticas con ruido gaussiano N(0, sigma^2)
17 ruido = np.random.normal(loc=0.0, scale=0.15, size=len(t))
18 y_medido = A @ c_teorico + ruido
19
20 print("--- 1. IDENTIFICACION DE PARAMETROS (LSTSQ) ---")
21 c_estimado, residuos, rango, s = la.lstsq(A, y_medido, rcond=None)
22 print(f"Coeficientes teoricos: {c_teorico}")
23 print(f"Coeficientes estimados: {np.round(c_estimado, 4)}")
24
25 print("\n--- 2. ANALISIS DE CONDICIONAMIENTO NUMERICO ---")
26 cond_A = la.cond(A)
27 cond_AtA = la.cond(A.T @ A)
28 print(f"Numero de condicion kappa(A) : {cond_A:.2f}")
29 print(f"Numero de condicion kappa(A^T A) : {cond_AtA:.2f} (Elevado al cuadrado)")
30
31 print("\n--- 3. EVALUACION DE RESIDUOS Y BONDAD DE AJUSTE (R^2) ---")
32 y_predicho = A @ c_estimado
33 e = y_medido - y_predicho
34 norma_residuo = la.norm(e)
35 SS_res = np.sum(e**2)
36 SS_tot = np.sum((y_medido - np.mean(y_medido))**2)
37 R2 = 1.0 - (SS_res / SS_tot)
38
39 print(f"Norma L2 del vector error: {norma_residuo:.4f}")
40 print(f"Coeficiente de determinacion R^2: {R2:.6f}")
41
42 print("\n--- 4. MATRIZ DE COVARIANZA DE LOS PARAMETROS ---")
43 # Varianza muestral del ruido residual
44 grados_libertad = len(t) - len(c_teorico)
45 sigma2_est = SS_res / grados_libertad
46 # Covarianza: sigma^2 * (A^T A)^(-1)
47 cov_c = sigma2_est * la.inv(A.T @ A)
48 desv_est = np.sqrt(np.diag(cov_c))
49
50 print("Matriz de covarianza de parametros:\n", np.round(cov_c, 5))
51 print("Incertidumbre estandar (1-sigma) en cada parametro:", np.round(desv_est, 4))

```

Listing 18: Script integral de ajuste matricial, estabilidad y análisis estadístico

Consola de Salida: Salida del script integrador

```

--- 1. IDENTIFICACION DE PARAMETROS (LSTSQ) ---
Coeficientes teoricos: [ 2.5 -1.8  0.75]
Coeficientes estimados: [ 2.5186 -1.8284  0.7562]

--- 2. ANALISIS DE CONDICIONAMIENTO NUMERICO ---

```

```

Numero de condicion kappa(A)      : 36.42
Numero de condicion kappa(A^T A) : 1326.43 (Elevado al cuadrado)

--- 3. EVALUACION DE RESIDUOS Y BONDAD DE AJUSTE (R^2) ---
Norma L2 del vector error: 0.2858
Coeficiente de determinacion R^2: 0.996184

--- 4. MATRIZ DE COVARIANZA DE LOS PARAMETROS ---
Matriz de covarianza de parametros:
[[ 0.01026 -0.01185  0.00262]
 [-0.01185  0.01777 -0.00438]
 [ 0.00262 -0.00438  0.00115]]
Incertidumbre estandar (1-sigma) en cada parametro: [0.1013 0.1333 0.0339]

```

[▲ Volver al índice general](#)

7. Buenas Prácticas, Rendimiento y Estabilidad Numérica

7.1. ¿Cuándo Utilizar numpy.linalg frente a scipy.linalg?

Aunque `numpy.linalg` cubre las operaciones esenciales, `scipy.linalg` es más amplio y contiene rutinas avanzadas:

- **Garantía de enlace a LAPACK/BLAS:** `scipy.linalg` siempre se compila con soporte de aceleradores completos.
- **Descomposiciones adicionales:** Factorizaciones especializadas como Schur, Hessenberg, matrices de Toeplitz, Cholesky por bandas y resolución de ecuaciones de Sylvester/Lyapunov.
- **Operaciones directas sobre matrices estructuradas:** Resolución directa de matrices tridiagonales o definidas positivas sin alojar ceros en memoria.

7.2. Disposición de Datos en Memoria: Orden C vs. Orden Fortran

- **C-contiguous** (por defecto en NumPy): Almacenamiento por filas contiguas (*Row-major*). Los elementos contiguos en memoria corresponden a la variación del último índice ($A[i, j]$ seguido de $A[i, j + 1]$).
- **Fortran-contiguous:** Almacenamiento por columnas (*Column-major*). Las rutinas clásicas de LAPACK están escritas originalmente para matrices en orden Fortran.

Si se trabaja con matrices enormes en bucles de cálculo, asegurarse de pasar matrices con contigüidad adecuada (`np.ascontiguousarray` o `np.asfortranarray`) previene copias intermedias implícitas en memoria.

7.3. Comparaciones Numéricas Seguras

Debido a la representación de punto flotante binario IEEE 754, jamás deben compararse números reales con el operador de igualdad estricta `==`. En su lugar, utilice siempre `np.isclose` o `np.allclose` especificando tolerancias absoluta y relativa:

```

1 # Incorrecto (peligroso por precision finita):
2 # if det_A == 0.0: ...
3
4 # Correcto y profesional:
5 if np.isclose(la.det(A), 0.0, atol=1e-12):

```

```
6 print("La matriz es singular dentro de la tolerancia numerica.")
```

Listing 19: Comparaciones flotantes seguras

[▲ Volver al índice general](#)

8. Tabla Resumen de Funciones y Complejidades

La Tabla 1 resume las funciones fundamentales de álgebra lineal en Python, detallando su módulo de procedencia, propósito analítico, complejidad algorítmica y recomendaciones de uso.

Función	Librería	Operación Matemática	Complejidad	Recomendación de Uso
$A @ B$	Lenguaje Python	Multiplicación matricial AB	$\mathcal{O}(n^3)$	Preferir sobre <code>np.dot</code> en 2D.
<code>la.det(A)</code>	<code>numpy.linalg</code>	Determinante $\det(A)$	$\mathcal{O}(n^3)$	Si $n \gg 50$, usar <code>la.slogdet</code> .
<code>la.inv(A)</code>	<code>numpy.linalg</code>	Matriz inversa A^{-1}	$\mathcal{O}(n^3)$	Evitar para resolver sistemas.
<code>la.solve(A, b)</code>	<code>numpy.linalg</code>	Resolver $Ax = b$	$\mathcal{O}(n^3)$	Método canónico exacto (LU).
<code>la.eig(A)</code>	<code>numpy.linalg</code>	Autovalores y autovectores	$\mathcal{O}(n^3)$	Matrices generales asimétricas.
<code>la.eigh(A)</code>	<code>numpy.linalg</code>	Diagonalización simétrica	$\mathcal{O}(n^3)$	Si $A = A^T$, usar siempre este.
<code>la.svd(A)</code>	<code>numpy.linalg</code>	Valores singulares $U\Sigma V^T$	$\mathcal{O}(mn^2)$	Compresión, PCA y rango.
<code>la.qr(A)</code>	<code>numpy.linalg</code>	Factorización ortogonal QR	$\mathcal{O}(mn^2)$	Mínimos cuadrados estables.
<code>la.cholesky(A)</code>	<code>numpy.linalg</code>	Cholesky $A = LL^T$	$\frac{1}{3}\mathcal{O}(n^3)$	Matrices simétricas def. pos.
<code>spla.lu(A)</code>	<code>scipy.linalg</code>	Factorización $PA = LU$	$\frac{2}{3}\mathcal{O}(n^3)$	Eliminación gaussiana base.
<code>la.norm(x)</code>	<code>numpy.linalg</code>	Norma vectorial/matricial	$\mathcal{O}(n)/\mathcal{O}(n^2)$	Evaluar magnitudes y residuo.
<code>la.matrix_rank(A)</code>	<code>numpy.linalg</code>	Rango numérico $\text{rang}(A)$	$\mathcal{O}(mn^2)$	Basado en SVD con tolerancia.
<code>la.pinv(A)</code>	<code>numpy.linalg</code>	Pseudoinversa Moore-Penrose	$\mathcal{O}(mn^2)$	Sistemas sin solución única.
<code>la.lstsq(A, b)</code>	<code>numpy.linalg</code>	Mínimos cuadrados $\ Ax - b\ _2$	$\mathcal{O}(mn^2)$	Ajuste óptimo sobredeterminado.
<code>la.cond(A)</code>	<code>numpy.linalg</code>	Número de condición $\kappa(A)$	$\mathcal{O}(n^3)$	Medir susceptibilidad a error.

Tabla 1: Compendio analítico y computacional de funciones de álgebra lineal en Python.

[▲ Volver al índice general](#)

9. Conclusiones y Referencias

9.1. Conclusiones

La implementación de operaciones de álgebra lineal en Python a través de `numpy.linalg` y `scipy.linalg` combina la alta expresividad del lenguaje con el rendimiento nativo de rutinas compiladas en C y Fortran (BLAS/LAPACK). Las directrices centrales para su uso efectivo son:

1. **Separación conceptual:** Mantener estricta diferenciación entre la aritmética de arreglos (producto de Hadamard elemento a elemento `*`) y el álgebra lineal vectorial y matricial (`@`, `solve`, `lstsq`).
2. **Estabilidad sobre formulación teórica:** Sustituir fórmulas teóricas de libro de texto que involucren inversiones explícitas A^{-1} o ecuaciones normales $A^T A$ por métodos factoriales estables (eliminación LU, descomposición QR o SVD).
3. **Selección informada de algoritmos:** Aprovechar propiedades estructurales de las matrices (simetría, positividad definida, esparsidad) para seleccionar rutinas optimizadas como `eigh` o `cholesky`, que reducen la complejidad a la mitad e incrementan la robustez numérica.

9.2. Referencias Bibliográficas

1. Strang, G. (2016). *Introduction to Linear Algebra* (5th ed.). Wellesley-Cambridge Press.
2. Golub, G. H., & Van Loan, C. F. (2013). *Matrix Computations* (4th ed.). The Johns Hopkins University Press.
3. Trefethen, L. N., & Bau, D. (1997). *Numerical Linear Algebra*. SIAM: Society for Industrial and Applied Mathematics.
4. Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). *Array programming with NumPy*. *Nature*, 585(7825), 357–362.
5. Virtanen, P., Gommers, R., Oliphant, T. E., et al. (2020). *SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python*. *Nature Methods*, 17(3), 261–272.

[▲ Volver al índice general](#)